

VNA Reference Plane Shifting using 12 term error model

Sipan Hovsepian

Abstract

In high-frequency electronics, the Vector Network Analyzer (VNA) is the primary instrument for characterizing the stochastic and deterministic behavior of RF components. However, raw data acquired at the VNA's test ports is rarely a true representation of the Device Under Test (DUT) due to systematic errors. This article details a Python-based implementation of a 12-term error model designed to shift the measurement reference plane through a 10dB attenuator and associated cabling. By de-embedding these components, we reveal the fundamental S-parameters of the DUT with high precision.

1. I. INTRODUCTION: TARGET AUDIENCE AND USEFULNESS

By default, a VNA measures at its front-panel connectors. To obtain accurate data, engineers must mathematically “shift” the reference plane to the DUT interface. This process requires identifying and removing systematic measurement errors—such as directivity, source match, and reflection tracking—that would otherwise mask the true performance of the component.

This article is primarily intended for undergraduate and graduate engineering students, as well as junior RF engineers seeking a deeper mathematical intuition for vector network analysis. By providing a transparent, step-by-step implementation, this work serves as a practical bridge between Signal Flow Graph analysis and the software-driven measurements encountered in modern laboratories, allowing readers to apply these concepts to their own custom de-embedding challenges.

2. II. MOTIVATION: BEYOND THE “BLACK BOX”

In modern commercial VNAs, error correction is often a “one-touch” process facilitated by automated calibration modules (ECal) and proprietary firmware. While this efficiency is essential for laboratory throughput, it often renders the underlying mathematical transformations opaque to the user.

The primary objective of this project is pedagogical. Relying on a VNA as a “black box” is sufficient for daily productivity, but mastering the mathematics behind the measurement is essential for advanced RF engineering. By manually coding error-correction algorithms, we can “look under the hood” of the calibration engine and observe how raw, uncorrected measurements are transformed into clean, usable data.

While existing literature typically covers abstract algebraic proofs or ‘how-to’ guides for vendor-specific VNA hardware, this work offers a middle ground through an open-source, reproducible computational framework. By utilizing the `scikit-rf` library to process real-world Touchstone data, it provides a functional template that moves beyond textbook idealizations into the realm of practical, scriptable computation.

This work is inspired by the contributions of Brian Walker of Copper Mountain Technologies, who provided a clear and practical treatment of one-port VNA calibration using the three-term error model Walker (2014). While Walker’s work focuses on single-port reflection measurements, the present study extends this framework to the two-port, twelve-term error model. This extension is implemented in a Python-based computational environment to manage the increased algebraic and matrix complexity involved in full two-port error correction.

3. III. ANALYTICAL FRAMEWORK AND TOOLCHAIN

To replicate the VNA’s internal calibration engine, this implementation synthesizes classical RF literature with modern data science tools:

- **Mathematical Foundation:** The formulation follows Douglas Rytting’s 2003 ARFTG paper, “Network Analyzer Error Models and Calibration Methods” Rytting (2003), which defines the full 12-term error model and its associated calibration procedures.
- **Computational Environment:** Python is used to implement the calibration routines, providing native support for complex-valued numerical operations and matrix-based computation.
- **Data Handling:** The `scikit-rf (skrf)` Arsenovic et al. (2022) library was utilized for data ingestion. Rather than developing a custom parser for Touchstone files, `scikit-rf` was used to ingest raw `.s1p` and `.s2p` files, allowing the project to focus entirely on the implementation of the error-correction math.

With the toolchain established, the first step is to define the measurement problem geometrically. The VNA’s default reference plane sits at its front-panel connectors; the objective of this work is to shift that plane past the 10 dB attenuator and associated cabling to the DUT interface, as illustrated in [Figure 1](#).

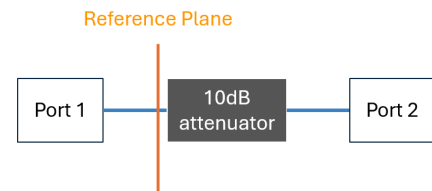
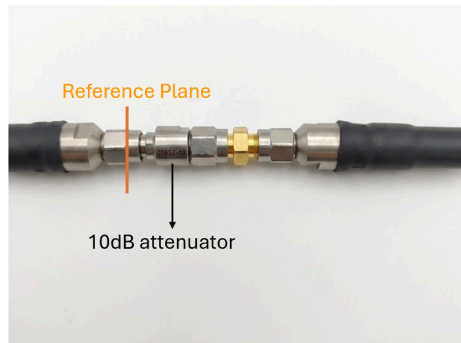


Figure 1: Default reference plane

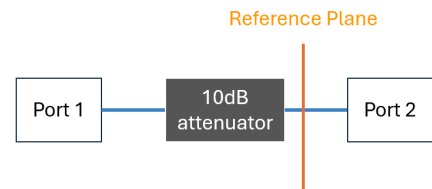
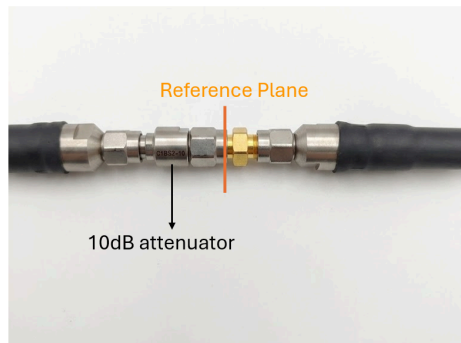


Figure 2: Desired Reference plane

4. THEORETICAL BACKGROUND

The 12-term error model consists of two separate 6-term models: one for the forward direction (Port 1) and another for the reverse direction (Port 2). Based on Douglas Rytting's formulations, the signal flow graphs for both models and the resulting equations used to correct the raw measured values are illustrated below.

12-Term Error Model

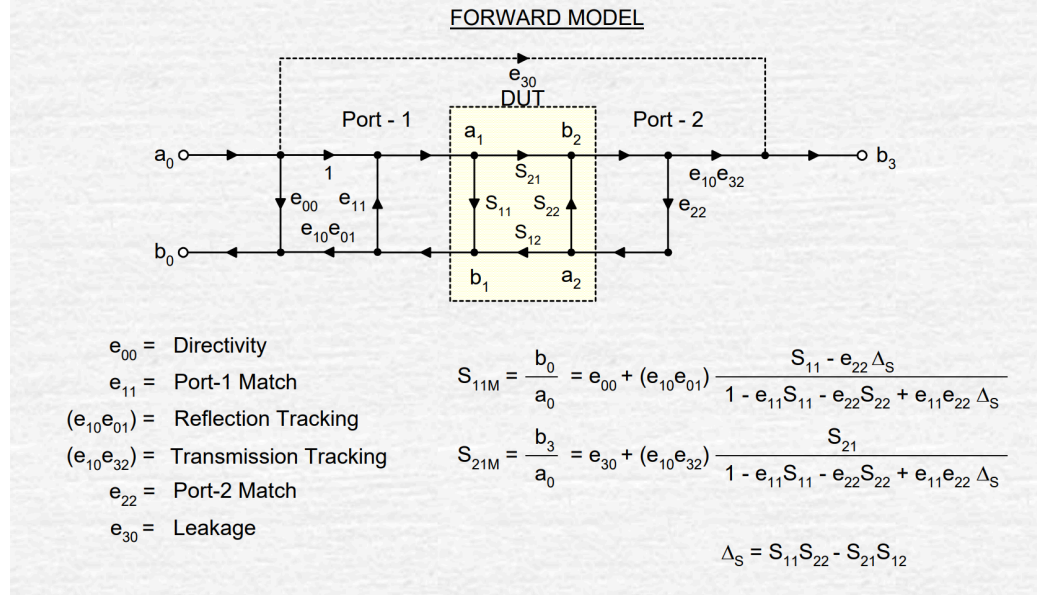


Figure 3: Forward Model

12-Term Error Model

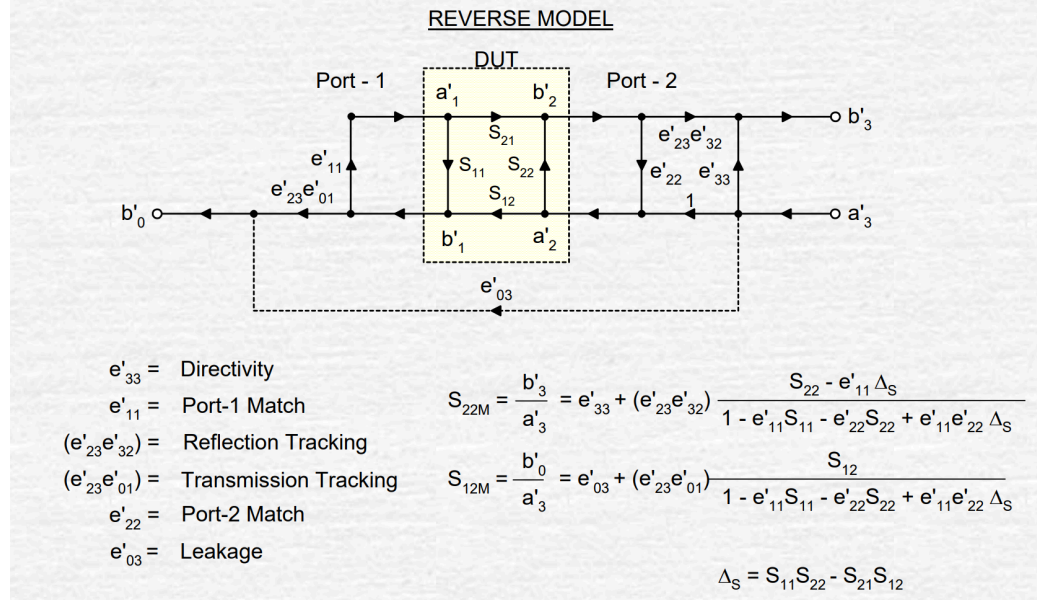


Figure 4: Reverse Model

$$\begin{aligned}
S_{11} &= \frac{\left(\frac{S_{11M} - e_{00}}{e_{10}e_{01}} \right) \left[1 + \left(\frac{S_{22M} - e'_{33}}{e'_{23}e'_{32}} \right) e'_{22} \right] - e_{22} \left(\frac{S_{21M} - e_{30}}{e_{10}e_{32}} \right) \left(\frac{S_{12M} - e'_{03}}{e'_{23}e'_{01}} \right)}{D} \\
S_{21} &= \frac{\left(\frac{S_{21M} - e_{30}}{e_{10}e_{32}} \right) \left[1 + \left(\frac{S_{22M} - e'_{33}}{e'_{23}e'_{32}} \right) (e'_{22} - e_{22}) \right]}{D} \\
S_{22} &= \frac{\left(\frac{S_{22M} - e'_{33}}{e'_{23}e'_{32}} \right) \left[1 + \left(\frac{S_{11M} - e_{00}}{e_{10}e_{01}} \right) e_{11} \right] - e'_{11} \left(\frac{S_{21M} - e_{30}}{e_{10}e_{32}} \right) \left(\frac{S_{12M} - e'_{03}}{e'_{23}e'_{01}} \right)}{D} \\
S_{12} &= \frac{\left(\frac{S_{12M} - e'_{03}}{e'_{23}e'_{01}} \right) \left[1 + \left(\frac{S_{11M} - e_{00}}{e_{10}e_{01}} \right) (e_{11} - e'_{11}) \right]}{D} \\
D &= \left[1 + \left(\frac{S_{11M} - e_{00}}{e_{10}e_{01}} \right) e_{11} \right] \left[1 + \left(\frac{S_{22M} - e'_{33}}{e'_{23}e'_{32}} \right) e'_{22} \right] - \left(\frac{S_{21M} - e_{30}}{e_{10}e_{32}} \right) \left(\frac{S_{12M} - e'_{03}}{e'_{23}e'_{01}} \right) e_{22}e'_{11}
\end{aligned}$$

Figure 5: Formulas

5. METHODOLOGY

The logic for acquiring the directivity, source match, and reflection tracking terms is identical to the standard 3-term error model. Because the 12-term model relies on independent forward and reverse models, these initial terms are extracted by performing a standard 1-port calibration (using Open, Short, and Load standards) on Port 1 and Port 2 separately. The remaining transmission terms (load match, transmission tracking, and isolation) are then solved using the Thru measurement.

• Step 1: One-Port Calibration on Port 1

- Calibrate Port-1 using the standard one-port procedure.
- Solve for e_{11} , e_{00} , and $(e_{10}e_{01})$.
- Calculate $(e_{10}e_{01})$ from Δ_e .

• Step 2: Isolation Measurement

- Connect Z_0 terminations to both Ports 1 and 2.
- Measure S_{21M} , which provides the isolation term (e_{30}) directly.

• Step 3: Thru Measurement

- Connect Ports 1 and 2 together.
- Calculate the load match (e_{22}) using:

$$e_{22} = \frac{S_{11M} - e_{00}}{S_{11M}e_{11} - \Delta_e} \quad (1)$$

- Calculate the transmission tracking $(e_{10}e_{32})$ using:

$$e_{10}e_{32} = (S_{21M} - e_{30})(1 - e_{11}e_{22}) \quad (2)$$

- **Step 4: Reverse Model**

- Use the exact same process for the reverse model to extract the remaining 6 terms.

5.a. Step 1

The following illustrations show the physical connections made to acquire the “ideal” and “measured” data, which are then processed by the Python script.

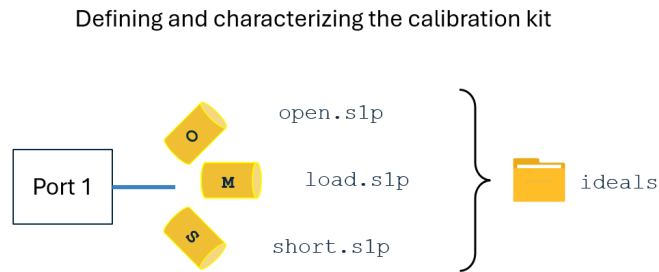


Figure 6: Configuration for “ideal”

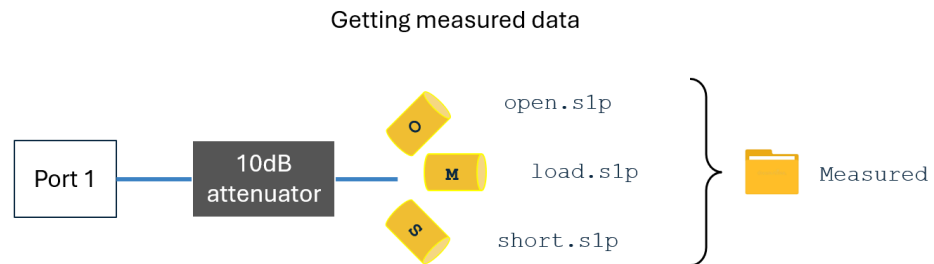


Figure 7: Configuration for “measured”

Using `scikit-rf`, the raw Touchstone files acquired from the setups above are imported and converted into Network objects, and calculations shown in [Figure 3](#) are performed.

```
import skrf as rf
import matplotlib.pyplot as plt
import numpy as np
import math

numpoints=1601;
#ideals
openGamma=rf.Network('Port1/ideals/open.slp')
shortGamma=rf.Network('Port1/ideals/short.slp')
loadGamma=rf.Network('Port1/ideals/load.slp')
#measured
m_openGamma=rf.Network('Port1/measured/open.slp')
m_shortGamma=rf.Network('Port1/measured/short.slp')
```

```

m_loadGamma=rf.Network('Port1/measured/load.slp')

# Gamma Actual

open_Gamma=openGamma.s
short_Gamma=shortGamma.s
load_Gamma=loadGamma.s

Gamma_a1=np.zeros((numpoints),dtype=complex)
Gamma_a2=np.zeros((numpoints),dtype=complex)
Gamma_a3=np.zeros((numpoints),dtype=complex)

Gamma_m1=np.zeros((numpoints),dtype=complex)
Gamma_m2=np.zeros((numpoints),dtype=complex)
Gamma_m3=np.zeros((numpoints),dtype=complex)

for i in range(0,numpoints):
    Gamma_a1[i]=complex(open_Gamma[i,0,0].real,open_Gamma[i,0,0].imag)
    Gamma_a2[i]=complex(short_Gamma[i,0,0].real,short_Gamma[i,0,0].imag)
    Gamma_a3[i]=complex(load_Gamma[i,0,0].real,load_Gamma[i,0,0].imag)
# Gamma Measured

m_open_Gamma=m_openGamma.s
m_short_Gamma=m_shortGamma.s
m_load_Gamma=m_loadGamma.s

for i in range(0,numpoints):
    Gamma_m1[i]=complex(m_open_Gamma[i,0,0].real,m_open_Gamma[i,0,0].imag)
    Gamma_m2[i]=complex(m_short_Gamma[i,0,0].real,m_short_Gamma[i,0,0].imag)
    Gamma_m3[i]=complex(m_load_Gamma[i,0,0].real,m_load_Gamma[i,0,0].imag)

A=np.zeros((3,3),dtype=complex)
B=np.zeros((3,3),dtype=complex)
X=np.zeros((3,1),dtype=complex)

e_00=np.zeros((numpoints),dtype=complex) # Directivity
e_11=np.zeros((numpoints),dtype=complex) #Source match
delta_e=np.zeros((numpoints),dtype=complex) #Delta e
e_10_e_01=np.zeros((numpoints),dtype=complex) #Reflection Tracking

for i in range (0,numpoints):
    A[0,0]=1
    A[0,1]=Gamma_a1[i]*Gamma_m1[i]
    A[0,2]=-Gamma_a1[i]
    A[1,0]=1
    A[1,1]=Gamma_a2[i]*Gamma_m2[i]
    A[1,2]=-Gamma_a2[i]
    A[2,0]=1
    A[2,1]=Gamma_a3[i]*Gamma_m3[i]

```

```

A[2,2]=-Gamma_a3[i]

B[0]=Gamma_m1[i]
B[1]=Gamma_m2[i]
B[2]=Gamma_m3[i]

A_inv=np.linalg.inv(A)
X=np.dot(A_inv,B)

e_00[i]=X[0,0]
e_11[i]=X[1,0]
delta_e[i]=X[2,0]
e_10_e_01[i]=e_00[i]*e_11[i]-delta_e[i]

np.save("variables/e_00", e_00)
np.save("variables/e_11", e_11)
np.save("variables/e_10_e_01", e_10_e_01)

```

The same process was repeated for Port 2.

After this step, we acquire 6 out of the 12 error terms:

- e_{00} (Forward Directivity)
- e_{11} (Forward Source Match)
- $e_{10}e_{01}$ (Forward Reflection Tracking)
- $\bar{e}_{33}$ (Reverse Directivity)
- $\bar{e}_{11}$ (Reverse Source Match)
- $\bar{e}_{23}\bar{e}_{32}$ (Reverse Reflection Tracking)

5.b. Step 2

Step 2, handles leakage. Both ports are match terminated, and S_{21M} and S_{12M} were saved as e_{30} and $\bar{e}_{03}$

The following figure illustrates the configuration.

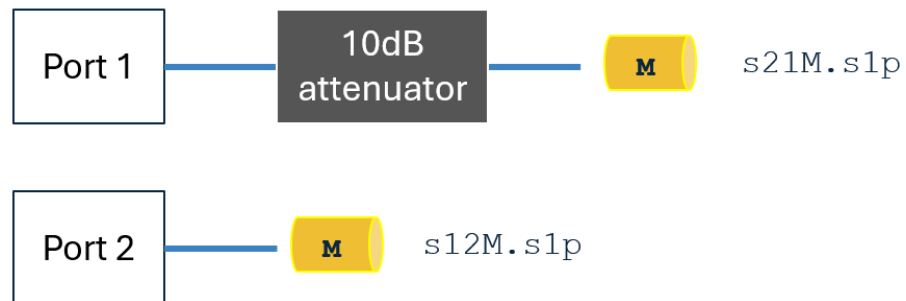


Figure 8: Configuration for Leakage measurements

```

import skrf as rf
import matplotlib.pyplot as plt
import numpy as np
import math

numpoints=1601;

s21_m=rf.Network('step2/s21M.slp')
s12_m=rf.Network('step2/s12M.slp')

s21M=s21_m.s
s12M=s12_m.s

s21_M=np.zeros((numpoints),dtype=complex)
s12_M=np.zeros((numpoints),dtype=complex)

for i in range(0,numpoints):
    s21_M[i]=complex(s21M[i,0,0].real,s21M[i,0,0].imag)
    s12_M[i]=complex(s12M[i,0,0].real,s12M[i,0,0].imag)

e_30=np.zeros((numpoints),dtype=complex) # leakage
e_03_bar=np.zeros((numpoints),dtype=complex)

for i in range(0,numpoints):
    e_30[i]=s21_M[i]
    e_03_bar[i]=s12_M[i]

np.save("variables/e_30", e_30)
np.save("variables/e_03_bar", e_03_bar)

```

After this step, we acquire 2 additional error terms:

- e_{30} (Forward Leakage)
- $\bar{e}_{03}$ (Reverse Leakage)

5.c. Step 3

Finally, we have only 4 error terms remaining, which are found by making the configuration displayed below.



Figure 9: Through connection

```

import skrf as rf
import matplotlib.pyplot as plt
import numpy as np
import math

numpoints=1601;

e_00=np.load("variables/e_00.npy")
e_11=np.load("variables/e_11.npy")
e_10_e_01=np.load("variables/e_10_e_01.npy")
e_30=np.load("variables/e_30.npy")
delta_e=np.zeros((numpoints),dtype=complex)

for i in range(0,numpoints):
    delta_e[i]=(e_00[i]*e_11[i])-e_10_e_01[i]

s11M=rf.Network('step3/s11M.slp')
s12M=rf.Network('step3/s12M.slp')
s21M=rf.Network('step3/s21M.slp')
s22M=rf.Network('step3/s22M.slp')

s11M=s11M.s
s12M=s12M.s
s21M=s21M.s
s22M=s22M.s

s11_M=np.zeros((numpoints),dtype=complex)
s12_M=np.zeros((numpoints),dtype=complex)
s21_M=np.zeros((numpoints),dtype=complex)
s22_M=np.zeros((numpoints),dtype=complex)

for i in range(0,numpoints):
    s11_M[i]=complex(s11M[i,0,0].real,s11M[i,0,0].imag)
    s12_M[i]=complex(s12M[i,0,0].real,s12M[i,0,0].imag)
    s21_M[i]=complex(s21M[i,0,0].real,s21M[i,0,0].imag)
    s22_M[i]=complex(s22M[i,0,0].real,s22M[i,0,0].imag)

e_22=np.zeros((numpoints),dtype=complex)

```

```

e_10_e_32=np.zeros((numpoints),dtype=complex)

for i in range (0,numpoints):
    e_22[i]=(s11_M[i]-e_00[i])/(s11_M[i]*e_11[i]-delta_e[i])
    e_10_e_32[i]=(s21_M[i]-e_30[i])*(1-e_11[i]*e_22[i])

np.save("variables/e_22", e_22)
np.save("variables/e_10_e_32", e_10_e_32)

```

The similar procedure is repeated for Reverse model.

After this step, we got all 4 remaining error terms:

- e_{22} (Forward Load Match)
- $e_{10}e_{32}$ (Forward Transmission Tracking)
- $\bar{e}_{22}$ (Reverse Load Match)
- $\bar{e}_{23}\bar{e}_{01}$ (Reverse Transmission Tracking)

6. CALCULATION

After acquiring, we load 12 error terms to do final set of calculations for correction, shown in [Figure 5](#)

```

import skrf as rf
import matplotlib.pyplot as plt
import numpy as np
import math

numpoints=1601;

#load s Measured

s11M=rf.Network('step3/s11M.slp')
s12M=rf.Network('step3/s12M.slp')
s21M=rf.Network('step3/s21M.slp')
s22M=rf.Network('step3/s22M.slp')

s11M=s11M.s
s12M=s12M.s
s21M=s21M.s
s22M=s22M.s

s11_M=np.zeros((numpoints),dtype=complex)
s12_M=np.zeros((numpoints),dtype=complex)
s21_M=np.zeros((numpoints),dtype=complex)
s22_M=np.zeros((numpoints),dtype=complex)

s11=np.zeros((numpoints),dtype=complex)
s12=np.zeros((numpoints),dtype=complex)
s21=np.zeros((numpoints),dtype=complex)

```

```

s22=np.zeros((numpoints),dtype=complex)

for i in range(0,numpoints):
    s11_M[i]=complex(s11M[i,0,0].real,s11M[i,0,0].imag)
    s12_M[i]=complex(s12M[i,0,0].real,s12M[i,0,0].imag)
    s21_M[i]=complex(s21M[i,0,0].real,s21M[i,0,0].imag)
    s22_M[i]=complex(s22M[i,0,0].real,s22M[i,0,0].imag)

# load 12 terms
e_22_bar=np.load("variables/e_22_bar.npy")
e_23_bar_e_01_bar=np.load("variables/e_23_bar_e_01_bar.npy")
e_10_e_32=np.load("variables/e_10_e_32.npy")
e_22=np.load("variables/e_22.npy")
e_00=np.load("variables/e_00.npy")
e_10_e_01=np.load("variables/e_10_e_01.npy")
e_11=np.load("variables/e_11.npy")
e_03_bar=np.load("variables/e_03_bar.npy")
e_30=e_03_bar=np.load("variables/e_30.npy")
e_11_bar=np.load("variables/e_11_bar.npy")
e_23_bar_e_32_bar=np.load("variables/e_23_bar_e_32_bar.npy")
e_33_bar=np.load("variables/e_33_bar.npy")

D=np.zeros((numpoints),dtype=complex)

for i in range(0,numpoints):
    D[i] = (
        (1 + ((s11_M[i] - e_00[i]) / e_10_e_01[i]) * e_11[i]) *
        (1 + ((s22_M[i] - e_33_bar[i]) / e_23_bar_e_32_bar[i]) *
e_22_bar[i]) -
        ((s21_M[i] - e_30[i]) / e_10_e_32[i]) *
        ((s12_M[i] - e_03_bar[i]) / e_23_bar_e_01_bar[i]) *
        e_22[i] * e_11_bar[i]
    )

for i in range(0,numpoints):
    s21[i] = (
        ((s21_M[i] - e_30[i]) / e_10_e_32[i]) * (1 + ((s22_M[i] - e_33_bar[i]) /
e_23_bar_e_32_bar[i]) * (e_22_bar[i] - e_22[i]))) / D[i]
    )

for i in range(0,numpoints):
    s21[i]=20*math.log10(abs(s21[i]))
    s21_M[i]=20*math.log10(abs(s21_M[i]))

```

```
plt.figure()

plt.plot(s21, label='Corrected S21')
plt.plot(s21_M, label='Raw S21')
plt.legend()
```

7. RESULTS

To test the program, we plotted the measured and corrected S_{21} data. The measurement setup includes a 10 dB attenuator. By mathematically moving the reference plane past the attenuator using our 12-term correction, we expect to offset the effects of this attenuation in the corrected data.

The following figure shows the final result.

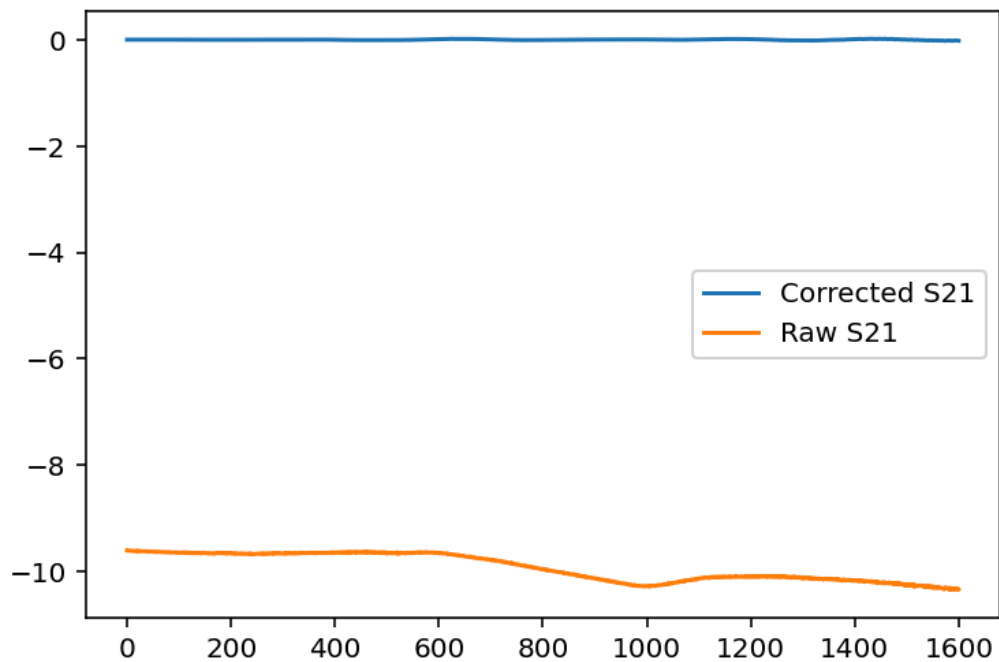


Figure 10: Results

The 12-term correction successfully de-embeds the attenuator. The corrected S_{21} is approximately 0 dB across the measured frequency range, recovering roughly 10 dB of insertion loss introduced by the attenuator and cabling. The result is consistent with expected behavior.

REFERENCES

- Arsenovic, A., Hillairet, J., Anderson, J., Forstén, H., Rieß, V., Eller, M., Sauber, N., Weikle, R., Barnhart, W., & Forstmayr, F. (2022). scikit-rf: An Open Source Python Package for Microwave Network Creation, Analysis, and Calibration [Speaker's Corner]. *IEEE Microwave Magazine*, 23(1), 98–105. <https://doi.org/10.1109/MMM.2021.3117139>

- Rytting, D. (2003,). Network Analyzer Error Models and Calibration Methods. *61st ARFTG Conference Short Course Notes*.
- Walker, B. (2014). One-Port VNA Calibration: A Look Under the Hood. *Microwaves & RF*. <https://www.mwrf.com/technologies/test-measurement/article/21152010/copper-mountain-technologies-one-port-vna-calibration-a-look-under-the-hood>